



TP N° 2 – MLP

Perceptron Multicouche (MLP)

COMPTE RENDU

Module	Apprentissage Automatique (DL)
Étudiante	Zaynab AIT ADDI
Professeur	Pr. Mohamed CHERRADI
Filière	TDIA2
Université	Abdelmalek Essaâdi — ENSA AI Hoceima
Année	2025 – 2026

Ce rapport présente les implémentations d'un Perceptron Multicouche (MLP) avec Keras/TensorFlow, appliquées à quatre exercices couvrant la classification multi-classes (MNIST, CIFAR-10), la classification binaire (Breast Cancer) et la régression (Boston Housing).

Introduction

Le TP N°2 porte sur le **Perceptron Multicouche (MLP)**, une extension naturelle du Perceptron Simple (SLP) étudié au TP N°1. Contrairement au SLP limité à la classification linéaire, le MLP est capable d'apprendre des représentations non linéaires complexes grâce à ses couches cachées et à la rétropropagation du gradient.

Principe du MLP

Un MLP est constitué de plusieurs couches denses entièrement connectées : une couche d'entrée, une ou plusieurs couches cachées avec activation non linéaire (ReLU), et une couche de sortie adaptée à la tâche. L'apprentissage repose sur la minimisation d'une fonction de perte par descente de gradient stochastique (SGD ou Adam) via la **rétropropagation**.

La règle de mise à jour des poids par rétropropagation s'écrit :

$$\begin{aligned} W_{\text{new}} &= W_{\text{old}} - \alpha \times \partial L / \partial W \\ b_{\text{new}} &= b_{\text{old}} - \alpha \times \partial L / \partial b \end{aligned}$$

où α = taux d'apprentissage, L = fonction de perte

Avantage du MLP sur le SLP : le MLP peut approximer n'importe quelle fonction continue (théorème d'approximation universelle) grâce à ses couches cachées non linéaires.

Pour chaque exercice, une architecture Keras Sequential est définie, compilée avec Adam et évaluée sur un jeu de test. Les données sont prétraitées (normalisation, one-hot encoding) selon les bonnes pratiques.

Exercice 01 – MLP sur MNIST

Objectif : Classifier les chiffres manuscrits (0 à 9) du dataset MNIST avec un MLP à deux couches cachées.

1.1 Description du dataset

Caractéristique	Valeur
Nombre d'images (train)	60 000
Nombre d'images (test)	10 000
Taille des images	28 × 28 pixels (niveaux de gris)
Vecteur d'entrée après aplatissement	784 attributs
Nombre de classes	10 (chiffres 0 à 9)

1.2 Prétraitement des données

Deux opérations essentielles ont été réalisées avant l'entraînement :

- Aplatissement : chaque image 28×28 est convertie en un vecteur de 784 valeurs.
- Normalisation : les pixels sont divisés par 255 pour obtenir des valeurs dans [0, 1], ce qui accélère la convergence.
- Encodage one-hot : les étiquettes entières (0–9) sont converties en vecteurs binaires de taille 10, requis par la perte categorical_crossentropy.

```
x_train = x_train.reshape(-1, 784).astype('float32') / 255.0
x_test  = x_test.reshape(-1, 784).astype('float32') / 255.0
y_train_cat = to_categorical(y_train, num_classes=10)
y_test_cat  = to_categorical(y_test, num_classes=10)
```

1.3 Architecture du modèle

Couche	Taille de sortie	Paramètres
Dense 128 – ReLU	(None, 128)	$784 \times 128 + 128 = 100\,480$
Dense 64 – ReLU	(None, 64)	$128 \times 64 + 64 = 8\,256$
Dense 10 – Softmax	(None, 10)	$64 \times 10 + 10 = 650$
TOTAL	—	109 386 paramètres

```
model = Sequential([
    Input(shape=(784,)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax'),
])
```

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

La couche de sortie utilise Softmax qui transforme les scores bruts en probabilités dont la somme vaut 1. La classe prédite est celle avec la probabilité maximale (argmax). La perte categorical_crossentropy mesure la divergence entre la distribution prédite et la vraie distribution one-hot.

1.4 Entraînement

Paramètres : 5 époques, batch size = 64, validation_split = 0.2 (20 % du train réservés à la validation).

Époque	Perte Train	Précision Train	Perte Val.	Précision Val.
1	0.3088	91.18 %	0.1594	95.36 %
2	0.1299	96.13 %	0.1273	96.22 %
3	0.0913	97.28 %	0.1030	97.06 %
4	0.0675	97.93 %	0.1088	96.70 %
5	0.0533	98.34 %	0.0899	97.39 %

1.5 Résultats & Analyse

Perte (test)	Précision (test)
0.0819	97.53 %

Analyse : Le MLP atteint 97.53 % de précision sur le jeu de test MNIST, ce qui représente une amélioration significative par rapport au SLP (~89 %) du TP1. Les couches cachées permettent d'apprendre des caractéristiques hiérarchiques (contours, formes locales). La perte de validation décroît régulièrement sans surapprentissage notable sur 5 époques, ce qui confirme la bonne généralisation du modèle.

***Comparaison avec le SLP (TP1)** : SLP MNIST $\approx$ 89 % vs MLP MNIST $\approx$ 97.5 %. Les deux couches cachées ReLU permettent d'apprendre des transformations non linéaires que le SLP ne peut pas capturer.*

Exercice 02 – MLP sur CIFAR-10

Objectif : Classifier les images couleur RGB 32×32 en 10 catégories d'objets avec un MLP plus profond.

2.1 Description du dataset

Caractéristique	Valeur
Images d'entraînement	50 000
Images de test	10 000
Dimensions	32 × 32 × 3 (RGB couleur)
Vecteur d'entrée	3 072 attributs
Classes (10)	avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion

2.2 Prétraitement

- Aplatissement : images (32, 32, 3) → vecteur de 3072 valeurs.
- Normalisation : pixels divisés par 255 → valeurs dans [0, 1].
- Encodage one-hot : étiquettes converties en vecteurs binaires de taille 10.

```
x_train = x_train.reshape(-1, 3072).astype('float32') / 255.0
x_test  = x_test.reshape(-1, 3072).astype('float32') / 255.0
y_train_cat = to_categorical(y_train, num_classes=10)
y_test_cat  = to_categorical(y_test, num_classes=10)
```

2.3 Architecture du modèle

Couche	Taille de sortie	Paramètres
Dense 512 – ReLU	(None, 512)	$3072 \times 512 + 512 = 1\,573\,376$
Dense 256 – ReLU	(None, 256)	$512 \times 256 + 256 = 131\,328$
Dense 10 – Softmax	(None, 10)	$256 \times 10 + 10 = 2\,570$
TOTAL	—	1 707 274 paramètres

```
model = Sequential([
    Input(shape=(3072,)),
    Dense(512, activation='relu'),
    Dense(256, activation='relu'),
    Dense(10, activation='softmax'),
])
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

2.4 Entraînement

Paramètres : 10 époques, batch size = 64, validation_split = 0.2.

Époque	Perte Train	Précision Train	Perte Val.	Précision Val.
1	1.9030	31.63 %	1.7543	37.30 %
3	1.6166	42.52 %	1.6370	41.58 %
5	1.5168	45.88 %	1.5267	45.58 %
7	1.4618	47.94 %	1.4927	46.28 %
10	1.3849	50.69 %	1.4637	47.85 %

2.5 Résultats & Analyse

Perte MSE (test)	Précision (test)
1.4707	47.68 %

Analyse : Le MLP n'atteint que 47.68 % sur CIFAR-10, bien en dessous des 97.53 % obtenus sur MNIST. Cette différence s'explique par plusieurs facteurs fondamentaux :

- Complexité visuelle : les images CIFAR-10 sont en couleur (3 canaux) et contiennent des variations importantes d'éclairage, de pose et d'arrière-plan, contrairement aux chiffres MNIST sur fond uniforme.
- Perte de la structure spatiale : l'aplatissement en vecteur 3072 détruit toute information spatiale locale. Un CNN conserverait les relations de voisinage entre pixels.
- Ambiguïté inter-classes : certaines classes (chien/chat, voiture/camion) sont visuellement très proches.

Limite du MLP sur CIFAR-10 : La précision ~47 % confirme que le MLP dense n'est pas adapté aux images couleur complexes. Les Réseaux de Neurones Convolutifs (CNN) atteignent >90 % sur ce même dataset en exploitant la localité et l'invariance par translation des filtres convolutifs.

Exercice 03 – MLP sur Breast Cancer Wisconsin

Objectif : Classification binaire des tumeurs (bénigne vs maligne) à partir de caractéristiques numériques extraites d'images de biopsie.

3.1 Description du dataset

Caractéristique	Valeur
Échantillons totaux	569
Attributs numériques	30 (rayon, texture, périmètre, aire, lissé...)
Classes	0 = Bénin, 1 = Malin (convention du TP)
Séparation	80 % train / 20 % test (stratifiée)

3.2 Prétraitement

Les 30 attributs présentent des échelles très différentes (rayon en mm vs aire en mm²). La **standardisation** (moyenne = 0, écart-type = 1) est donc indispensable pour que toutes les caractéristiques contribuent équitablement à l'apprentissage.

```
# Convention : 0 = Bénin, 1 = Malin (inverse de sklearn)
y = (1 - data.target).astype('float32')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train).astype('float32')
X_test = scaler.transform(X_test).astype('float32')
```

Important : le `StandardScaler` est ajusté uniquement sur le train, puis appliqué au test. Cela évite toute fuite d'information du test vers le train (data leakage).

3.3 Architecture du modèle

Couche	Taille de sortie	Paramètres
Dense 64 – ReLU	(None, 64)	$30 \times 64 + 64 = 1\,984$
Dense 32 – ReLU	(None, 32)	$64 \times 32 + 32 = 2\,080$
Dense 1 – Sigmoid	(None, 1)	$32 \times 1 + 1 = 33$
TOTAL	—	4 097 paramètres

```
model = Sequential([
    Input(shape=(30,)),
    Dense(64, activation='relu'),
```

```
Dense(32, activation='relu'),
Dense(1, activation='sigmoid'), # sortie binaire
])
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

La couche de sortie utilise la sigmoïde $\sigma(x) = 1/(1+e^{-x})$ qui produit une probabilité $p \in [0, 1]$. Si $p > 0.5 \rightarrow$ classe 1 (Malin), sinon classe 0 (Bénin). La perte `binary_crossentropy` est la mesure standard pour la classification binaire.

3.4 Entraînement

Paramètres : 10 époques, batch size = 32.

Époque	Perte Train	Précision Train
1	0.4550	83.30 %
2	0.2451	92.97 %
4	0.1329	95.60 %
6	0.0968	96.70 %
8	0.0808	97.36 %
10	0.0681	98.02 %

3.5 Résultats & Analyse

Perte (test)	Précision (test)
0.0719	99.12 %

Analyse : Le modèle atteint une précision exceptionnelle de 99.12 % sur le jeu de test. Ce résultat confirme que le MLP est très bien adapté aux données tabulaires numériques normalisées. La convergence est rapide (dès l'époque 3, la précision dépasse 94 %) et la perte continue de décroître de manière régulière jusqu'à 0.0681, signe d'un apprentissage stable.

- La standardisation joue un rôle crucial : sans elle, les attributs à grande échelle (aire) domineraient les gradients.
- Le jeu de données, bien que petit (569 échantillons), est suffisant pour ce réseau compact de 4 097 paramètres.
- Une seule erreur de classification sur ~114 exemples de test illustre la robustesse du modèle.

Application médicale : un tel modèle, même s'il ne remplace pas le diagnostic clinique, peut servir d'outil d'aide à la décision pour signaler les cas suspects à examiner en priorité.

Exercice 04 – MLP pour la Régression (Boston Housing)

Objectif : Prédire le prix médian des logements (en milliers de dollars) à partir de 13 caractéristiques socio-économiques et géographiques.

4.1 Description du dataset

Caractéristique	Valeur
Nombre d'échantillons	506 (404 train / 102 test, via Keras)
Attributs	13 (criminalité, accessibilité, nb pièces, distance emploi...)
Variable cible	Prix médian en milliers de dollars (valeur continue)
Type de tâche	Régression (sortie réelle)

4.2 Prétraitement

Les 13 attributs ont des échelles très hétérogènes. La normalisation est appliquée en soustrayant la moyenne et en divisant par l'écart-type, calculés **exclusivement sur le train** pour éviter toute fuite d'information.

```
(x_train, y_train), (x_test, y_test) = boston_housing.load_data()

mean = x_train.mean(axis=0)
std = x_train.std(axis=0)
std_safe = np.where(std == 0, 1e-8, std) # éviter division par zéro

x_train = ((x_train - mean) / std_safe).astype('float32')
x_test = ((x_test - mean) / std_safe).astype('float32')
```

4.3 Architecture du modèle

Couche	Taille de sortie	Paramètres
Dense 64 – ReLU	(None, 64)	$13 \times 64 + 64 = 896$
Dense 32 – ReLU	(None, 32)	$64 \times 32 + 32 = 2\,080$
Dense 1 – Linéaire	(None, 1)	$32 \times 1 + 1 = 33$
TOTAL	—	3 009 paramètres

```
model = Sequential([
    Input(shape=(13,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1, # activation linéaire (régression)
])
model.compile(optimizer='adam',
```

```
loss='mean_squared_error',
metrics=[RootMeanSquaredError(name='rmse')])
```

La couche de sortie n'a pas de fonction d'activation (linéaire par défaut), permettant de prédire n'importe quelle valeur réelle. La perte MSE pénalise fortement les grandes erreurs. La RMSE est plus interprétable car elle est dans la même unité que la variable cible (k\$).

4.4 Entraînement

Paramètres : 50 époques, batch size = 32.

Époque	Perte MSE Train	RMSE Train (k\$)
1	592.69	24.34
5	399.09	19.98
10	88.20	9.39
20	26.13	5.11
30	18.37	4.29
40	14.16	3.76
50	11.63	3.41

4.5 Résultats & Analyse

Métrique	Valeur (test)	Interprétation
MSE	21.78 (k\$)²	Erreur quadratique moyenne
RMSE	4.67 k\$	Erreur moyenne $\approx \pm 4\,670$ \$

Analyse : Le modèle converge efficacement sur 50 époques, avec la RMSE passant de 24.34 k\$ (époque 1) à 3.41 k\$ (train) et 4.67 k\$ (test). Une RMSE de ~ 4.67 k\$ signifie que le modèle prédit le prix d'une maison avec une erreur d'environ $\pm 4\,670$ \$, ce qui est raisonnable compte tenu de la complexité du marché immobilier.

- L'écart entre RMSE train (3.41 k\$) et RMSE test (4.67 k\$) indique un léger surapprentissage, compréhensible sur un dataset aussi petit (404 exemples d'entraînement).
- La décroissance très rapide de la MSE (de 592 à 88 entre les époques 1 et 10) confirme que l'optimiseur Adam est bien adapté à ce type de problème.
- L'activation linéaire de la couche de sortie est la différence clé par rapport aux exercices de classification : aucune borne sur la valeur prédite.

Régression vs Classification : en régression, on n'utilise ni Softmax ni Sigmoid en sortie. La perte MSE mesure l'écart au carré entre valeur prédite et réelle, et la RMSE (sa racine) est dans la même unité que la cible, donc directement interprétable.

Conclusion Générale

Ce TP a permis d'explorer les capacités du **Perceptron Multicouche (MLP)** sur quatre problèmes représentatifs de l'apprentissage profond. En voici une synthèse comparative :

Exercice	Dataset	Tâche	Architecture	Résultat (test)
Ex.01	MNIST	Classification 10 classes	784→128→64→10 (Softmax)	97.53 %
Ex.02	CIFAR-10	Classification 10 classes	3072→512→256→10 (Softmax)	47.68 %
Ex.03	Breast Cancer	Classification binaire	30→64→32→1 (Sigmoid)	99.12 %
Ex.04	Boston Housing	Régression	13→64→32→1 (Linéaire)	RMSE = 4.67 k\$

Enseignements clés

- Le MLP surpasse nettement le SLP sur MNIST (97.5 % vs ~89 %), grâce à ses couches cachées non linéaires capables d'apprendre des représentations intermédiaires.
- Pour les images complexes (CIFAR-10), le MLP dense est insuffisant (~47 %) car il ignore la structure spatiale. Les CNN sont adaptés à ce cas.
- Sur des données tabulaires (Breast Cancer, Boston Housing), le MLP excelle à condition de normaliser correctement les entrées (StandardScaler ou normalisation manuelle).
- Le choix de la fonction d'activation de sortie et de la perte est dicté par la tâche : Softmax + categorical_crossentropy (multi-classes), Sigmoid + binary_crossentropy (binaire), Linéaire + MSE (régression).
- La standardisation des données (moyenne 0, écart-type 1) est indispensable pour les données tabulaires et améliore systématiquement la vitesse de convergence et les performances finales.

Perspectives

Pour améliorer les résultats obtenus, plusieurs pistes peuvent être explorées : l'ajout de couches de **Dropout** pour réduire le surapprentissage, l'utilisation de **CNN** pour CIFAR-10, l'augmentation du nombre d'époques, ou encore l'application de techniques de **data augmentation** pour enrichir le jeu d'entraînement.

Ce TP illustre que le MLP est un modèle polyvalent et puissant pour les données structurées, mais que les architectures spécialisées (CNN pour images, RNN pour séquences) restent nécessaires pour les données non tabulaires complexes.